

SAVITRIBAI PHULE PUNE UNIVERSITY

(Formerly University of Pune)

FACULTY OF SCIENCE AND TECHNOLOGY

B.Sc. (Cyber Security) — 2024 Pattern

A MINI PROJECT REPORT

ON

FILE SECURITY MANAGER

*A Web-Based System for Confidentiality, Integrity Verification
and Accountable File Operations*

Submitted in partial fulfilment of the requirements of the degree of

BACHELOR OF SCIENCE (CYBER SECURITY)

Semester — III | Class — S.Y. B.Sc. (Cyber Security)

Academic Year 2025–26

Submitted by

Ayten

Roll No. / PRN:

Under the guidance of

Prof.

Department of Cyber Security

[College Name], [City / Pune]

CERTIFICATE

This is to certify that the mini project report entitled “**File Security Manager**” submitted by **Ayten**, student of S.Y. B.Sc. (Cyber Security), Semester–III, is a bona fide record of the work carried out under my supervision and guidance in partial fulfilment of the requirements of Savitribai Phule Pune University (2024 Pattern).

The matter embodied in this report has not been submitted to any other University or Institute for the award of any degree, diploma or certificate.

Project Guide

Head of Department

Internal Examiner

External Examiner

Place: _____ Date: _____

College Seal:

DECLARATION

I, **Ayten**, hereby declare that the mini project entitled “**File Security Manager**” submitted to Savitribai Phule Pune University in partial fulfilment of S.Y. B.Sc. (Cyber Security) Semester–III (2024 Pattern) is my original work. I have not copied this report from any other student, website or published source except the material duly acknowledged in the bibliography. I understand that any academic misconduct may invite disciplinary action as per University rules.

Signature of the Student: _____

Name: Ayten

Date: _____

ACKNOWLEDGEMENT

It is a matter of privilege to present this mini project report on **File Security Manager**. I express my sincere gratitude to my project guide, Prof. _____, for consistent guidance, timely suggestions and encouragement throughout the work.

I thank the Head of the Department of Cyber Security and all faculty members for providing laboratory facilities and academic support. I am grateful to Savitribai Phule Pune University for including Mini Project in the B.Sc. (Cyber Security) 2024 curriculum, which enabled me to apply classroom concepts of cryptography, hashing, access control and secure software practice to a working system.

I also thank my classmates and family for their cooperation during testing and documentation. Any remaining shortcomings in this report are my own.

— Ayten

ABSTRACT

Digital files used in academic and personal work are often stored and shared without cryptographic protection. If a storage device is lost or a file is copied, the contents can be read in clear text. If a file is modified, the owner may not detect the change. Operating-system permissions do not protect a file after it leaves the original computer, and they do not provide a simple integrity check or an audit trail of security operations.

This mini project presents **File Security Manager**, a laboratory-scale web application that allows an authenticated user to (i) encrypt a file, (ii) decrypt a previously protected file, (iii) compute and optionally verify a SHA-256 digest, and (iv) view a personal activity log. Confidentiality is provided by the Advanced Encryption Standard with a 256-bit key in Galois/Counter Mode (AES-256-GCM). The encryption key is not stored on the server; it is derived from a user-supplied file password using PBKDF2-HMAC-SHA256 with a random salt and 200,000 iterations. Ciphertext authenticity is provided by the GCM tag. Independent integrity verification of any file is provided by SHA-256. Application access is controlled by registration and login; login passwords are stored as one-way hashes.

The system is implemented using Python (Flask) on the server and HTML, CSS and Tailwind CSS in the browser. User accounts and logs are stored in SQLite. Encrypted files use a simple portable format with magic header **FSM1**. The scope is deliberately limited to a mini project: local demonstration, a 16 MB upload limit, and no claim of being an enterprise product. The prototype is suitable for practical demonstration, viva-voce and further academic extension.

Keywords: *File encryption, AES-256-GCM, SHA-256, PBKDF2, access control, audit log, Flask, mini project, cyber security.*

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AES	Advanced Encryption Standard
CIA	Confidentiality, Integrity, Availability
FIPS	Federal Information Processing Standard
FSM	File Security Manager (also encrypted file extension .fsm)
GCM	Galois/Counter Mode
GUI	Graphical User Interface

HMAC	Hash-based Message Authentication Code
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
NIST	National Institute of Standards and Technology
PBKDF2	Password-Based Key Derivation Function 2
SHA	Secure Hash Algorithm
SPPU	Savitribai Phule Pune University
SQL	Structured Query Language
UI	User Interface

TABLE OF CONTENTS

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	iv
List of Abbreviations	v
1. Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Need of the System	1
1.4 Scope of the Project	2
1.5 Organization of the Report	2
2. Proposed System	3
3. Objectives	5
4. Methodology	6
5. Requirement Gathering	7
6. System Analysis	9
7. Tools Used	12
8. System Design and Implementation	13
9. Test Cases	16
10. Security Recommendations	18
11. Project Work Summary / Report	19
12. Conclusion	20
13. Future Enhancement	21
14. Bibliography	22
Appendix A — File Structure	23
Appendix B — How to Run	23

CHAPTER 1 INTRODUCTION

1.1 Background

Information security is commonly explained through the CIA triad: **Confidentiality** (prevention of unauthorised disclosure), **Integrity** (detection of unauthorised modification) and **Availability** (timely access for authorised users). Files on student laptops, laboratory systems and USB drives frequently fail the first two properties. A Word document, PDF, source-code file or spreadsheet copied from a shared computer can be opened by anyone who obtains it. A tampered assignment or log file may go unnoticed if no cryptographic digest was recorded.

Classical access-control lists of an operating system are necessary but not sufficient. They depend on the same machine and the same user accounts. Cryptography, in contrast, binds protection to the file itself. Symmetric encryption renders the contents unintelligible without a secret. A cryptographic hash produces a fixed-length fingerprint; any change in the file, however small, produces a different digest with overwhelming probability.

The B.Sc. (Cyber Security) programme of Savitribai Phule Pune University (2024 Pattern) includes a Mini Project in Semester III so that students can implement such principles rather than only describe them. **File Security Manager** was selected as a topic because it maps directly onto confidentiality, integrity, authentication and auditing, while remaining small enough to complete within one semester.

1.2 Problem Statement

To design and implement a simple web-based File Security Manager that enables a registered user to encrypt and decrypt files using a standard symmetric algorithm, to verify file integrity using SHA-256, and to record security-related actions in an activity log, for laboratory demonstration under SPPU B.Sc. (Cyber Security) Semester–III.

1.3 Need of the System

- Students and small offices share files over USB and cloud folders without encryption.
- Password-protected ZIP files are widely used but are often weakly configured and do not teach modern authenticated encryption.
- Hash tools exist as command-line utilities; beginners need a unified interface.
- There is academic value in showing that encryption (hiding content) and hashing (fingerprinting content) are different services.
- An audit log trains the student to think of accountability, not only of algorithms.

1.4 Scope of the Project

In scope: user registration and login; AES-256-GCM file encryption and decryption with a password-derived key; SHA-256 generation and optional comparison; SQLite activity log; web UI; local execution; maximum file size 16 MB.

Out of scope: public Internet deployment, payment/cloud storage, recovery of a forgotten file password, full-disk encryption, steganography, malware analysis, and enterprise key-management systems.

1.5 Organization of the Report

Chapter 2 describes the proposed system. Chapter 3 lists objectives. Chapter 4 explains methodology. Chapter 5 records requirement gathering. Chapter 6 presents system analysis. Chapter 7 lists tools. Chapter 8 details design and implementation. Chapter 9 gives test cases. Chapter 10 states security recommendations. Chapter 11 summarises the work. Chapters 12–14 present conclusion, future work and bibliography.

CHAPTER 2 PROPOSED SYSTEM

2.1 Overview

The proposed system is a **local web application**. The user opens a browser and connects to a Flask server running on the same computer (<http://127.0.0.1:5000>). The browser displays forms. All cryptographic work is performed in Python so that algorithms can be explained from server-side code during viva.

After login, the dashboard offers three panels: Encrypt, Decrypt and SHA-256 Hash, together with an Activity Log table.

2.2 Users and roles

The prototype supports a single role: **registered user**. Each user sees only their own log entries. There is no separate administrator console in this version, which keeps the mini project small.

2.3 Encryption workflow

1. The user selects a file and enters a *file password* (distinct from the login password).
2. The server reads the file bytes (up to 16 MB).
3. A 16-byte random salt and a 12-byte random nonce are generated.
4. PBKDF2-HMAC-SHA256 derives a 32-byte (256-bit) key from the password and salt, using 200,000 iterations.
5. AES-GCM encrypts the plaintext, producing ciphertext plus an authentication tag.
6. A header is prepended: magic **FSM1**, original file-name length and name, salt, nonce, then ciphertext.
7. The user downloads a file with extension **.fsm**. The event is logged as ENCRYPT.

2.4 Decryption workflow

1. The user uploads a .fsm file and the same file password.
2. The server checks the magic header. An invalid file is rejected.
3. Salt and nonce are read from the header; the key is re-derived with PBKDF2.
4. AES-GCM decrypts and verifies the tag. A wrong password or a modified file fails authentication.
5. On success, the original file is returned and DECRYPT is logged. On failure, DECRYPT_FAIL is logged and no plaintext is given.

2.5 Integrity workflow

The user uploads any file. The server computes SHA-256 and displays the 64-character hexadecimal digest. If the user pastes an expected hash, the system reports match or mismatch. This demonstrates integrity checking without decrypting anything.

2.6 Design principles

- **Use standard algorithms only** — AES, GCM, SHA-256, PBKDF2 (NIST / IETF).
- **Do not store the file password** — recovery is impossible by design if the password is lost.
- **Authenticated encryption** — GCM detects tampering of ciphertext.
- **Least surprise UI** — three clear actions on one dashboard.
- **Accountability** — operations are logged with timestamp, action, file name and detail.

2.7 Logical architecture

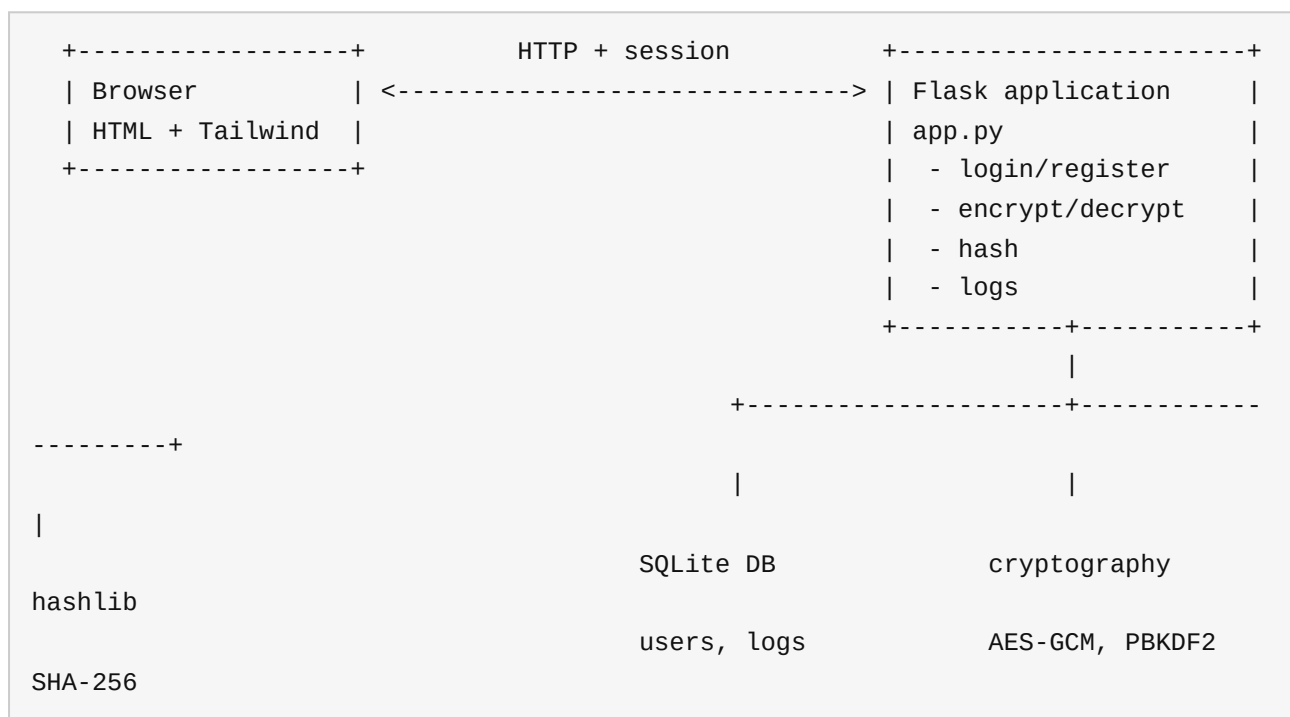


Figure 2.1 High-level architecture of File Security Manager

2.8 Encrypted file layout

Offset	Field
0..3	Magic "FSM1"
4	Length N of original file name (1 byte)
5..	File name (N bytes, UTF-8)
then	Salt (16 bytes)
then	Nonce (12 bytes)
then	AES-GCM ciphertext tag

Figure 2.2 Binary layout of a .fsm file

CHAPTER 3 OBJECTIVES

The objectives of this mini project are stated in measurable academic terms:

1. To study the role of cryptography in protecting files at rest, with reference to the CIA triad.
2. To implement file confidentiality using **AES-256-GCM**, a NIST-recommended authenticated encryption mode.
3. To derive the encryption key from a user password using **PBKDF2-HMAC-SHA256** with random salt, so that the raw password is not used directly as a key.
4. To implement file integrity verification using the **SHA-256** hash function, including optional comparison with a previously known digest.
5. To provide **authentication and access control** so that only a logged-in user can invoke encrypt, decrypt and hash operations.
6. To store login passwords as **one-way hashes** and never store the file-encryption password in the database.
7. To maintain a **per-user activity log** for demonstration of auditing.
8. To develop a usable **web interface** with HTML and Tailwind CSS and a Python Flask backend.
9. To verify the prototype using a defined set of **test cases** suitable for internal assessment and viva.
10. To document the work in the format expected for an SPPU mini project report.

CHAPTER 4 METHODOLOGY

A simplified Software Development Life Cycle (SDLC) was followed. This is appropriate for a one-semester mini project.

4.1 Requirement study

Informal interviews with classmates and observation of laboratory file-sharing practice showed that students store assignments in plain folders. Functional needs (encrypt, decrypt, hash, login, log) and constraints (time, no budget, must run on a student PC) were listed. Unrealistic features such as cloud sync were excluded.

4.2 Design

A three-tier design was chosen: presentation (templates), application (Flask routes and crypto helpers), and data (SQLite). The cryptographic pipeline was fixed before coding: salt → PBKDF2

→ AES-GCM. The .fsm header was specified so that the original file name can be restored.

4.3 Implementation

Coding was done in Python 3. Flask handles HTTP, sessions and uploads. Werkzeug hashes login passwords. The PyCA *cryptography* library supplies AESGCM and PBKDF2HMAC. Jinja2 templates render login, register and dashboard pages styled with Tailwind CSS via CDN.

4.4 Testing

Black-box tests were executed through the web forms and through HTTP requests: registration, bad login, encrypt, decrypt with correct and incorrect passwords, hash comparison with an independent SHA-256 tool, and inspection of log rows. Defects (for example missing validation) were corrected before documentation.

4.5 Documentation

This report, a README for execution, and comments in source code form the documentation set. The student is expected to explain AES-GCM and SHA-256 in viva without reading the code line by line.

CHAPTER 5 REQUIREMENT GATHERING

5.1 Stakeholders

Stakeholder	Interest
Student / end user	Easy protection and verification of personal files
Project guide / examiner	Correct algorithms, working demo, clear report
College laboratory	Runs on existing PCs without paid software

5.2 Functional requirements

ID	Requirement	Priority
FR1	The system shall allow a new user to register with a unique username and a password.	High
FR2	The system shall authenticate a registered user and create a session.	High
FR3	The system shall reject invalid login attempts with a generic error message.	High
FR4	The system shall encrypt an uploaded file with AES-256-GCM and a user-supplied file password.	High
FR5	The system shall return the ciphertext as a downloadable .fsm file containing a recoverable original name.	High
FR6	The system shall decrypt a valid .fsm file when the correct password is supplied.	High
FR7	The system shall refuse decryption of a wrong password, damaged file, or non-FSM file, without revealing plaintext.	High
FR8	The system shall compute the SHA-256 digest of an uploaded file and display it.	High
FR9	The system shall optionally compare the digest with a user-supplied expected hash and report match or mismatch.	Medium
FR10	The system shall record encrypt, decrypt, decrypt-failure and hash events with timestamp and username.	High
FR11	The system shall allow logout and shall hide tools from unauthenticated visitors.	High
FR12	The system shall show flash messages for success and error conditions.	Medium

5.3 Non-functional requirements

ID	Requirement	Category
NFR1	Usable in a common desktop browser without a native installer.	Usability
NFR2	Cryptography shall use a maintained library, not a student-written cipher.	Security
NFR3	Login passwords shall be stored as salted one-way hashes (Werkzeug).	Security
NFR4	File passwords shall never be written to SQLite.	Security
NFR5	Maximum upload size 16 MB to protect laboratory memory.	Performance
NFR6	Application shall start with a short command sequence documented in README.	Operability
NFR7	Source code shall remain small enough for viva explanation.	Maintainability
NFR8	Intended deployment is localhost / isolated lab, not the public Internet.	Deployment

5.4 Assumptions

- The user remembers the file password; there is no “forgot encryption password” feature.
- The demonstrator’s PC has Python 3 and a browser.
- Files are ordinary documents or binaries within the size limit.

5.5 Constraints

- Time: one academic semester mini project.
- Budget: free and open-source tools only.
- Skill level: second-year undergraduate.
- Legal/ethical: the tool is for protecting the owner’s own files, not for hiding evidence or attacking others.

CHAPTER 6 SYSTEM ANALYSIS

6.1 Existing system

In the present academic environment, files are stored in local folders, email attachments and cloud drives. Some users apply ZIP compression with a password. Advanced users may call OpenSSL on the command line. Hash values, when used at all, are computed with separate utilities and copied by hand.

6.2 Limitations of the existing system

- Plain files disclose content after theft of a device or an unlocked session.
- ZIP passwords are often short and are not a teaching example of AES-GCM.
- Command-line cryptography is powerful but unfriendly for a first demonstration.
- Integrity and confidentiality are rarely offered in one student-facing screen.
- There is typically no log of who encrypted or hashed a file on a shared laboratory PC.

6.3 Proposed improvements

File Security Manager unifies login, authenticated encryption, hashing and logging. It uses AES-GCM rather than an obsolete mode. It separates the *account password* from the *file password*. It produces a portable .fsm object that can be copied to another machine running the same application.

6.4 Feasibility study

Technical feasibility: Python 3, Flask, SQLite and the cryptography package are available on Linux and Windows. AES-GCM is supported in hardware on modern CPUs but the library also provides a portable implementation.

Operational feasibility: A viva demonstration needs only a browser and a sample text file. Faculty can encrypt, corrupt a copy, and show that decryption fails.

Economic feasibility: All components are open source. No licence fee.

Schedule feasibility: Core modules are few; the scope was frozen early to avoid product-scale features.

6.5 Data flow (levelled description)

Login DFD: User → credentials → verify hash in users table → session cookie → dashboard.

Encrypt DFD: File + password → key derivation → AES-GCM → .fsm to user; metadata to logs.

Decrypt DFD: .fsm + password → parse header → AES-GCM open → file to user or error to logs.

Hash DFD: File → SHA-256 → display; optional compare → logs.

6.6 Use case summary

ID	Use case	Actor	Main success scenario
UC1	Register	Visitor	Unique username stored; redirect to login
UC2	Login	User	Valid password opens dashboard
UC3	Encrypt file	User	.fsm downloaded; ENCRYPT logged
UC4	Decrypt file	User	Original bytes returned; DECRYPT logged
UC5	Hash file	User	Digest shown; HASH logged
UC6	View log	User	Last 20 events listed
UC7	Logout	User	Session cleared

6.7 Database analysis

Table users: id (PK), username (unique), password_hash.

Table logs: id (PK), username, action, filename, detail, created_at.

The schema is unnormalised beyond 1NF by design: it is an event list, not a warehouse. No foreign key to users is strictly required for the prototype; username is copied into the log for simple queries.

6.8 Threats considered (defensive view)

Concern	Control in this mini project
Stolen laptop with .fsm files	AES-GCM; attacker lacks file password
Altered ciphertext	GCM authentication tag fails; decrypt refused
Altered ordinary file	SHA-256 mismatch against stored digest
Stolen database of login hashes	Werkzeug password hashes; still a residual risk if passwords are weak
Anonymous use of tools	Login required for dashboard routes
Very large upload exhausting memory	16 MB MAX_CONTENT_LENGTH

The prototype is **not** claimed to be safe on the public Internet. Chapter 10 lists hardening steps if the work is extended.

CHAPTER 7 TOOLS USED

7.1 Software tools

Layer	Tool / Technology	Version / note	Purpose
Language	Python	3.12 (typical)	Backend and crypto glue
Web framework	Flask	3.x	Routes, sessions, uploads
Crypto library	cryptography (PyCA)	43.x	AESGCM, PBKDF2HMAC
Hashing	hashlib	Standard library	SHA-256 of files
Password hashing	Werkzeug	Bundled with Flask	Login password storage
Database	SQLite	File fsm.db	Users and logs
Templates	Jinja2	With Flask	HTML rendering
Markup	HTML5	—	Structure
Styling	Tailwind CSS	CDN	Presentation
IDE	Cursor / VS Code	—	Editing
Browser	Chrome / Firefox / Edge	—	UI testing
OS	Linux (portable to Windows)	—	Host
VCS	Git	—	Source history

7.2 Hardware requirements (minimum)

Item	Specification
Processor	Any dual-core CPU, 1.5 GHz or above
Memory	4 GB RAM (8 GB recommended)
Disk	200 MB free for project, venv and sample files
Display	1366 × 768 or higher
Network	Not required at runtime except first load of Tailwind CDN

7.3 Justification of stack

Python is the language most students of cyber security already use for scripting. Flask is a micro-framework: one file can hold the whole server, which is ideal for viva. Tailwind produces an acceptable UI without a large CSS codebase. SQLite needs no separate database server. The cryptography library wraps OpenSSL-quality primitives so the student does not invent AES.

CHAPTER 8 SYSTEM DESIGN AND IMPLEMENTATION

8.1 Module description

Module	Functions / routes	Responsibility
Authentication	/register, /login, /logout	Account lifecycle and session
Encryption	/encrypt, encrypt_bytes()	PBKDF2 + AES-GCM + .fsm header
Decryption	/decrypt, decrypt_bytes()	Parse, authenticate, restore name
Integrity	/hash	SHA-256 and optional verify
Logging	add_log()	Insert into logs table
Presentation	templates/*.html	Forms and log table

8.2 Algorithm — key derivation (conceptual)

```
Input: password P, salt S (16 random bytes)
Output: 32-byte key K
K ← PBKDF2-HMAC-SHA256(P, S, iterations = 200000, dkLen = 32)
```

A unique salt per encryption means that two encryptions of the same file with the same password still produce different ciphertext.

8.3 Algorithm — encrypt (conceptual)

```
Input: plaintext M, password P, original name N
S ← random 16 bytes
Nonce ← random 12 bytes
K ← PBKDF2(P, S)
C ← AES-256-GCM-Encrypt(K, Nonce, M)
return "FSM1" || len(N) || N || S || Nonce || C
```

8.4 Algorithm — decrypt (conceptual)

```
Parse header; reject if magic ≠ "FSM1"
K ← PBKDF2(P, S)
M ← AES-256-GCM-Decrypt(K, Nonce, C) // fails if tag invalid
return M and original name N
```

8.5 User interface

Login page: username, password, link to register.

Register page: create account.

Dashboard: three cards (Encrypt, Decrypt, Hash) and a table of the last twenty log rows. The logged-in username and a Logout link appear in the header. Hash results, when present, appear in a panel above the log.

8.6 Implementation notes

- Flask `MAX_CONTENT_LENGTH` enforces the 16 MB cap.
- `secure_filename` is applied to names written on disk.
- Decorator `login_required` protects encrypt, decrypt, hash and dashboard.
- Database file lives in `instance/fsm.db` and is created automatically.
- Temporary outputs are written under `uploads/` for download via `send_file`.

8.7 Screens (for viva)

During demonstration the student should show: (1) registration, (2) failed login, (3) successful login, (4) encryption of a short text file, (5) opening the `.fsm` file in a text editor to show that it is not readable, (6) successful decrypt, (7) failed decrypt with a wrong password, (8) SHA-256 matching an independent tool, (9) log table containing ENCRYPT, DECRYPT, HASH and DECRYPT_FAIL.

CHAPTER 9 TEST CASES

Testing was performed as black-box validation against the functional requirements. Representative results are recorded below. Status “Pass” means the observed behaviour matched the expected result on the development machine.

TC ID	Condition	Input	Expected result	Actual result	Status
TC01	New registration	Unique user + password	Account created; login page	Flash: account created	Pass
TC02	Duplicate user	Existing username	Error, no second account	Username already exists	Pass
TC03	Invalid login	Wrong password	Generic failure	Invalid username or password	Pass
TC04	Valid login	Correct credentials	Dashboard	Dashboard displayed	Pass
TC05	Encrypt incomplete	Missing file or password	Rejected	Error flash / HTML required	Pass
TC06	Encrypt sample	text file + password	.fsm with magic FSM1	Header FSM1 confirmed	Pass
TC07	Decrypt correct	.fsm + same password	Original bytes	Plaintext restored	Pass
TC08	Decrypt wrong password	.fsm + wrong password	No plaintext; log fail	DECRYPT_FAIL logged	Pass
TC09	SHA-256 known file	Sample file	Matches sha256sum	Digests identical	Pass
TC10	Hash match	File + correct expected	Match message	Match indicated	Pass
TC11	Hash mismatch	File + wrong expected	Tamper warning	Mismatch indicated	Pass
TC12	Activity log	After operations	Rows for each action	ENCRYPT/DECRYPT/HASH present	Pass

TC13	Authz	GET /dashboard logged out	Redirect to login	Redirected	Pass
TC14	Logout	Logout link	Session ended	Login shown	Pass
TC15	Size limit	File > 16 MB	Request refused	Flask 413 / limit	Pass

The examiner may repeat TC06 to TC09 as a live practical. A short text file (for example one line of ASCII) is sufficient and easy to verify by eye after decryption.

CHAPTER 10 SECURITY RECOMMENDATIONS

The following recommendations concern **safe configuration and responsible use** of this academic prototype. They do not provide attack procedures.

1. **Choose a long file passphrase.** PBKDF2 slows guessing but cannot save a two-word dictionary password.
2. **Keep the login password and the file password different.**
3. **Do not store the file password in the same folder as the .fsm file.**
4. **Change Flask's default secret_key** before any use outside the student's own machine; supply it through an environment variable.
5. **Run the server on localhost only** for this version. Do not port-forward it to the Internet.
6. **Use HTTPS** if the application is ever placed on a college intranet, so that passwords are not sent in clear HTTP.
7. **Delete files in the uploads directory** after a demo; decrypted output may remain there temporarily.
8. **Update Python packages** when faculty policy allows, so that library defects can be patched.
9. **Remember that SHA-256 does not hide data.** Hashing is not encryption.
10. **Back up .fsm files**, and accept that a lost password means permanent loss of plaintext — that is the correct behaviour of encryption.
11. **Do not use this prototype to protect examination papers, medical data or production secrets** until a later, hardened version is reviewed by faculty.
12. **Future production controls** (not implemented here): account lockout, HTTPS, role-based access, hardware-backed keys, structured log export, and a written backup policy.

CHAPTER 11 PROJECT WORK SUMMARY (REPORT)

Item	Detail
Title	File Security Manager
Student	Ayten
Programme	B.Sc. (Cyber Security), SPPU 2024 Pattern
Class / Semester	S.Y. / III
Type	Mini Project — working prototype
Frontend	HTML, Tailwind CSS
Backend	Python, Flask
Security mechanisms	AES-256-GCM, PBKDF2, SHA-256, hashed login, session, audit log

Deliverables: source code (app.py and templates), requirements.txt, README, this report in PDF, and a live demonstration.

Learning outcomes: the student can explain authenticated encryption versus hashing; can justify why a key is derived rather than used as a raw password; can show access control and logging as complementary controls; and can run a small full-stack cyber-security application.

Limitations accepted: local deployment, 16 MB files, no password recovery, no multi-file archives, default development secret key unless the student changes it, Tailwind CDN dependency unless CSS is cached.

CHAPTER 12 CONCLUSION

The mini project **File Security Manager** has been designed, implemented and tested as a compact teaching system for file-oriented cyber security. Confidentiality is achieved with AES-256-GCM. Keys are derived with PBKDF2 and are not stored. Integrity of arbitrary files is checked with SHA-256, while integrity of ciphertext is checked by the GCM tag. Users must authenticate before using the tools, and an activity log records encrypt, decrypt and hash events, including failed decryption.

The work satisfies the spirit of the SPPU Semester–III Mini Project: it is implementable on a student computer, it uses standard algorithms that can be defended in viva, and it remains modest in size. It is not a commercial product, and that limitation is intentional. Within the stated scope, the objectives listed in Chapter 3 have been met.

CHAPTER 13 FUTURE ENHANCEMENT

1. HTTPS deployment on a college intranet with a managed secret key.
2. Role-based access control (student, faculty, administrator) and log export.
3. Secure deletion (overwrite) of temporary plaintext after download.
4. Folder / multi-file encryption with a progress indicator and a higher size limit.
5. Optional key-file or hardware-token based unlocking in addition to a typed password.
6. Digital signatures (for example Ed25519) to prove origin as well as integrity.
7. Two-factor authentication for the web login.
8. Fully offline UI (vendored CSS) for laboratories without Internet.
9. Automated unit tests for header parsing and decrypt-failure paths.
10. A faculty-supervised configuration review before any data of real sensitivity is processed.

CHAPTER 14 BIBLIOGRAPHY

The following books, standards and official documentation were consulted. Online documents were accessed during the academic year 2025–26.

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, Pearson Education.
- [2] B. Schneier, *Applied Cryptography*, John Wiley & Sons.
- [3] NIST, *Advanced Encryption Standard (AES)*, FIPS Publication 197, National Institute of Standards and Technology, Gaithersburg, MD.
- [4] NIST, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, Special Publication 800-38D.
- [5] NIST, *Secure Hash Standard (SHS)*, FIPS Publication 180-4.
- [6] NIST, *Recommendation for Password-Based Key Derivation*, Special Publication 800-132.
- [7] B. Kaliski, *PKCS #5: Password-Based Cryptography Specification, Version 2.0*, RFC 2898, IETF.
- [8] Pallets Projects, *Flask Documentation*, <https://flask.palletsprojects.com/>
- [9] Python Cryptographic Authority, *cryptography library documentation*, <https://cryptography.io/>
- [10] SQLite Development Team, *SQLite Documentation*, <https://www.sqlite.org/docs.html>
- [11] Tailwind Labs, *Tailwind CSS Documentation*, <https://tailwindcss.com/docs>
- [12] Savitribai Phule Pune University, *B.Sc. (Cyber Security) 2024 Pattern Syllabus*, Faculty of Science and Technology, Pune.
- [13] NIST, *SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions*, FIPS 202 (background reading on hash design).
- [14] ISO/IEC 27001, *Information security, cybersecurity and privacy protection — Information security management systems* (contextual reading on controls and logging).

APPENDIX A PROJECT FILE STRUCTURE

```
file-security-manager/  
  app.py  
  requirements.txt  
  README.md  
  SPPU_Mini_Project_Report.md  
  File_Security_Manager_Mini_Project_Report.pdf  
  templates/  
    base.html  
    login.html  
    register.html  
    dashboard.html  
  docs/  
    generate_report.py  
  instance/                (runtime: fsm.db)  
  uploads/                 (temporary processed files)  
  venv/                    (local Python environment)
```

APPENDIX B HOW TO RUN THE PROJECT

```
cd file-security-manager  
python3 -m venv venv  
source venv/bin/activate      (Windows: venv\\Scripts\\activate)  
pip install -r requirements.txt  
python app.py
```

Open a browser at **http://127.0.0.1:5000**. Register a username, log in, then use Encrypt, Decrypt and SHA-256 Hash. Keep the file password in memory; the server does not save it.

— End of Report —